JAVASCRIPT • MODULE 06

Async/Await & Promises

Write Asynchronous JavaScript Cleanly

■ ASYNC JS

■ LEARNING OUTCOME

By the end of this module you will understand the JavaScript event loop, write clean async code with Promises and async/await, handle errors, and run parallel operations efficiently.

■ Skills You Will Gain

- Understand the event loop, call stack, and microtask queue
- Create and use Promises with .then/.catch/.finally
- Write clean async/await with try/catch
- Handle async errors correctly
- Run parallel operations with Promise.all
- Use Promise.allSettled, Promise.race, Promise.any

■ Every real web app fetches data, reads files, or queries databases. Without async JavaScript, your UI freezes while waiting. Async/await makes concurrent operations as readable as synchronous code.

SECTION 1

Event Loop & Promises

Why JS Needs Async -- The Single-Thread Problem

JavaScript runs on a SINGLE THREAD -- it can only do one thing at a time. If you run a slow operation synchronously, the ENTIRE PAGE freezes. The Event Loop solution: Call Stack: Where JS executes code -- one at a time Web APIs: Browser handles slow tasks (fetch, setTimeout) Microtask Queue: Promise callbacks -- higher priority Task Queue: setTimeout/setInterval callbacks Microtasks (Promises) always run BEFORE regular tasks (setTimeout).

```
// Execution order demonstration console.log('1 -- sync start'); setTimeout(() => console.log('4 -- setTimeout'), 0); Promise.resolve().then(() => console.log('3 -- Promise microtask')); console.log('2 -- sync end'); // Output: 1, 2, 3, 4 (Promise fires before setTimeout!) // CREATING PROMISES function delay(ms) { return new Promise((resolve, reject) => { if (ms < 0) reject(new Error('Negative delay')); setTimeout(() => resolve(`Done after ${ms}ms`), ms); }); } // Promise states: pending -> fulfilled (resolve called) or rejected (reject called) // Using .then / .catch / .finally (older style) delay(1000) .then(result => result.toUpperCase()) .then(upper => console.log(upper)) .catch(err => console.error('Failed:', err.message)) .finally(() => hideLoadingSpinner()); // always runs
```

SECTION 2

async/await -- The Modern Way

```
// async function ALWAYS returns a Promise async function fetchUser(userId) { try { // await pauses THIS function (doesn't block other code) const response = await fetch(`/api/users/${userId}`); if (!response.ok) { throw new Error(`HTTP ${response.status}: ${response.statusText}`); } const user = await response.json(); return user; } catch (error) { console.error('Fetch failed:', error.message); throw error; // re-throw -- let caller handle } finally { hideLoadingSpinner(); // always runs } } // PARALLEL -- run multiple at once (much faster!) // Sequential (BAD): total time = 3s + 2s + 1s = 6s async function loadSlow(userId) { const user = await fetchUser(userId); // 3s const orders = await fetchOrders(userId); // 2s const prefs = await fetchPrefs(userId); // 1s return { user, orders, prefs }; } // Parallel (GOOD): total time = 3s (longest) async function loadFast(userId) { const [user, orders, prefs] = await Promise.all([ fetchUser(userId), // all 3 start at SAME TIME fetchOrders(userId), fetchPrefs(userId) ]); return { user, orders, prefs }; } // Promise.allSettled -- wait for all, even if some fail const results = await Promise.allSettled([p1, p2, p3]); results.forEach(r => { if (r.status === 'fulfilled') use(r.value); else logError(r.reason); }); // Promise.race -- first to settle wins (timeout pattern) const withTimeout = (promise, ms) => Promise.race([ promise, new Promise((_,rej) => setTimeout(() => rej(new Error('Timeout')), ms)) ]); // Promise.any -- first to RESOLVE wins (ignore rejections) const fastest = await Promise.any([cdn1, cdn2, cdn3]);
```

■ PRACTICE Q & A

Q1. What is the difference between synchronous and asynchronous code?

A: *Synchronous: executes line by line, each waits for previous -- blocks thread. Asynchronous: starts operation, continues to next line, handles result later via Promise -- doesn't block.*

Q2. Does await block the JavaScript thread?

A: *No. await pauses only the async FUNCTION it's in. The event loop continues handling other tasks while waiting. Other code, event listeners, and UI updates keep working.*

Q3. What is the difference between Promise.all and Promise.allSettled?

A: *Promise.all: fails fast -- if ANY rejects, all fail. Promise.allSettled: waits for ALL regardless -- each result has {status:'fulfilled',value} or {status:'rejected',reason}. Use allSettled when partial success is OK.*

Q4. Why should you always have try/catch with async operations?

A: *Unhandled Promise rejections crash Node.js apps and show warnings in browsers. Always handle failures or your app silently breaks in production.*

Q5. What is the microtask queue and why do Promises fire before setTimeout?

A: *Promise callbacks go to the microtask queue which is processed BEFORE the task queue (setTimeout). The event loop processes ALL microtasks after each task, before picking the next task.*

■ Async/Await Cheat Sheet	
`async function name(){}`	Declare async function
`await promise`	Wait for Promise (inside async fn)
`try { await } catch(e){}`	Handle async errors
`new Promise((res,rej)=>{})`	Create Promise manually
`Promise.resolve(value)`	Immediately resolved
`Promise.reject(new Error())`	Immediately rejected
`Promise.all([p1,p2,p3])`	Parallel -- fails if any reject
`Promise.allSettled([...])`	Parallel -- waits for all
`Promise.race([p1,p2])`	First to settle wins
`Promise.any([p1,p2])`	First to RESOLVE wins
`.then(fn).catch(fn)`	Promise chain
`.finally(fn)`	Always runs -- cleanup